Estacionamiento inteligente con ESP32

TEI201 · Taller de Diseño en Ingeniería · Universidad Adolfo Ibáñez
Versión corregida: indicador encendido significa OCUPADO. Documento basado en el desarrollo y pruebas descritos en el chat y en la pauta oficial del Avance #2.

1. Problema, solución y alcance real

Problema: los estudiantes deben recorrer sectores del estacionamiento de la universidad para encontrar un cupo libre, generando vueltas y tiempo de búsqueda. Propuesta: un sensor por plaza detecta presencia de un vehículo; un controlador procesa el dato y activa un indicador visual. En una versión escalable, se sumarían los estados de varias plazas y se informaría el número de cupos por sector.

Estado Alpha probado en esta conversación: un ESP32, un sensor ultrasónico HC-SR04 y un segmento de un display de dos dígitos. Cuando hay objeto a menos de 15 cm, la rayita se enciende; cuando no lo hay a esa distancia, se apaga. No está implementado un dígito “1” completo, ni el conteo de dos plazas, ni comunicación inalámbrica.

2. Requisitos de la guía del Avance #2

La pauta pide demostración presencial de 5–7 min y 2 min de feedback, participación de todos, funcionamiento continuo al menos 5 min, sensor–actuador–conectividad integrada, modelo 3D conceptual del encapsulado, iteraciones verificables y documentación técnica.

Criterio de pauta	Peso	Situación según el chat
Funcionalidad Alpha	36 %	Sensor e indicador reportados funcionando; test de 5 min no documentado.
Integración	21 %	Sensor → ESP32 → display probado; transmisión Wi-Fi/Bluetooth pendiente.
Encapsulado 3D	14 %	Idea de carcasa discutida; modelo dimensional no aportado.
Iteración / testing	14 %	Cambios y fallos reales descritos durante montaje.
Documentación	7 %	Código y conexiones recopilados; diagrama gráfico por completar.
Comunicación	7 %	Exposición y roles de equipo aún por ensayar.

Las ponderaciones publicadas suman 99 % (por redondeo en la pauta). El modelo 3D no necesita estar impreso, pero sí presentado y justificado.

3. Componentes y materiales

Componente	Uso / estado
ESP32 Dev Module, 38 pines	Programado en Arduino IDE; monitor serie a 115200 baudios. Botón BOOT defectuoso.
HC-SR04, sensor 1	Instalado y validado; TRIG GPIO18, ECHO GPIO19 protegido.
HC-SR04, sensor 2	Adquirido posteriormente, pero no integrado por limitaciones de cableado y acceso a pines.
Kingbright DA56-11HDB	Display de dos dígitos; solo se controla una rayita del dígito derecho.
Protoboards	Blanca: circuito principal; transparente: display.
Resistencias	Tres de 10 kΩ para divisor de ECHO y una de 10 kΩ para la rayita. Había 100 kΩ y 220 kΩ; 220 kΩ no son 220 Ω.
Cables	Macho-macho disponibles; macho-hembra insuficientes para expandir. ESP32 queda fuera de la protoboard.

4. Conexiones del montaje final de una plaza

Origen	Destino / función
HC-SR04 VCC	5 V del ESP32, mediante protoboard.
HC-SR04 GND	GND común con ESP32.
HC-SR04 TRIG	GPIO18.
HC-SR04 ECHO	GPIO19 a través del divisor resistivo; nunca directamente.
Divisor ECHO	10 kΩ desde ECHO a GPIO19; 20 kΩ (2 × 10 kΩ en serie) de GPIO19 a GND. Reduce 5 V nominales a ≈3,33 V.
GPIO25	A23 de la protoboard transparente: control del ánodo común ensayado.
Segmento encendido	Pin del display en fila B20 → resistencia 10 kΩ → B12/A12 → GND del ESP32.

Conexión final del LED: GPIO25 → A23 → display → B20 → [10 kΩ] → B12/A12 → GND. GPIO25 en HIGH enciende; en LOW apaga. A23 no debe conectarse también a 3V3 cuando lo gobierna GPIO25. Las filas/columnas corresponden a la disposición concreta que se logró encender y son dependientes del montaje.

5. Código final coherente: rayita ENCENDIDA = OCUPADO

El siguiente es el código de referencia del chat que se decidió mantener. No requiere librerías adicionales distintas del entorno Arduino para ESP32.

```
#define TRIG 18
#define ECHO 19
#define LED_DISPLAY 25

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(LED_DISPLAY, OUTPUT);
  digitalWrite(LED_DISPLAY, LOW);
}

void loop() {
  digitalWrite(TRIG, LOW);
```

```
  delayMicroseconds(5);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long tiempo = pulseIn(ECHO, HIGH, 30000);
  if (tiempo == 0) {
    digitalWrite(LED_DISPLAY, LOW);
    Serial.println("Sin lectura");
  } else {
    float distancia = tiempo * 0.0343 / 2;
    if (distancia < 15) {
      digitalWrite(LED_DISPLAY, HIGH);
      Serial.println("OCUPADO");
    } else {
      digitalWrite(LED_DISPLAY, LOW);
      Serial.println("LIBRE");
    }
  }
  delay(500);
}
```

Medición	Resultado real del indicador
Distancia válida menor a 15 cm	OCUPADO → GPIO25 HIGH → rayita encendida.
Distancia válida de 15 cm o más	LIBRE → GPIO25 LOW → rayita apagada.
Sin eco (timeout)	Sin lectura → LOW → rayita apagada; no equivale a “plaza libre” confirmada.

Aclaración: el montaje final usa una sola barra del display como testigo de ocupación, no dibuja el número 1. Hubo una propuesta intermedia de invertir el indicador para que encendido significara LIBRE; esa versión fue descartada por el usuario y no se presenta aquí como final.

6. Iteración y testing documentados

Problema / decisión	Iteración efectivamente realizada
Programación ESP32	Botón BOOT defectuoso: puente temporal GPIO0–GND para carga; retirado al ejecutar.
Detección de objetos	Pruebas en Monitor Serie con objeto cercano y alejado; se validó la decisión de ocupación a umbral de 15 cm.
Protección eléctrica	Se instaló divisor para señal ECHO a 5 V utilizando 3 resistencias de 10 kΩ.
Montaje del display	El display no encajaba en protoboard pequeña. Se cambió a protoboard transparente con patas D20–D28 y G20–G28.
Pinout / conexión	Se encendieron segmentos uno por uno y se comprobó que una conexión fuera de fila impedía iluminación. Ajuste de resistencia a B12–B20 restauró la señal.
Simplificación	Por escasez de cables, se decidió utilizar una sola rayita en lugar del número 1 completo.
Integración final	Se pasó de alimentar el segmento con 3V3 fijo a controlarlo mediante GPIO25; se reportó funcionamiento de la indicación de ocupado.
Segundo sensor	Se recibió un segundo HC-SR04, pero se optó por presentar solo el montaje de un sensor por falta de cables hembra y exposición de pines macho.

No documentado aún: tabla cuantitativa de error/precisión, número de repeticiones, prueba de estabilidad durante 5 minutos, registro fotográfico fechado y transmisión inalámbrica. Conviene medirlos, no inventarlos.

7. Limitaciones concretas del prototipo

Conectores: el ESP32 de 38 pines tiene pines macho expuestos y se encuentra fuera de las protoboards. Los cables macho-macho no se pueden conectar de forma segura directamente a esos pines. Las dos protoboards disponibles y el ancho de la placa dificultan dejar hileras libres a cada lado. Los pocos cables macho-hembra se usaron para el HC-SR04 y el display. Por eso el segundo sensor no se añadió.

Visualización: el indicador solo enciende un segmento, no muestra el dígito 1 ni el total de cupos. Alimentación: probado por USB; una fuente USB de 5 V externa o power bank es una alternativa aún no confirmada por ensayo. Robustez: protoboards y cables Dupont pueden soltarse durante el transporte y necesitan fijación y protección. Escala: un único HC-SR04 detecta una zona frente a sí, no cuenta plazas generales de la universidad.

8. Márgenes de mejora: incorporación del segundo HC-SR04

Objetivo: pasar de una a dos plazas monitoreadas. El ESP32 recibiría dos mediciones de distancia, determinaría el estado ocupado/libre de cada espacio y calcularía disponibles = 2 − número de plazas ocupadas. Ese conteo podría mostrarse en un display apropiado o enviarse por Wi-Fi. Esta es una propuesta futura, no una capacidad construida.

Subtarea	Propuesta técnica / requisito
Conexión física	Conseguir cables Dupont hembra o adaptador/header accesible; alternativamente reorganizar montaje con protoboard compatible o placa soldable. No unir pines macho sueltos a la fuerza.
Sensor 2	VCC al suministro de 5 V y GND a la masa común; TRIG sugerido GPIO32, ECHO sugerido GPIO33. Pines a confirmar libres según montaje físico.
Protección ECHO 2	Segundo divisor propio: 10 kΩ desde ECHO a GPIO33 y 20 kΩ de GPIO33 a GND, igual al primer sensor.
Software	Añadir pulso/lectura independiente para el segundo sensor; calcular dos booleanos de ocupación y cupos disponibles. Medir en secuencia para reducir interferencia ultrasónica.
Interfaz	Pasar de rayita indicadora a un número 0, 1 o 2; requerirá identificar pinout real del Kingbright, resistencias apropiadas por segmento y suficientes salidas/cableado o driver.
Calibración y estabilidad	Ajustar umbral por geometría real, usar histéresis, filtrar lecturas y tratar timeout como estado desconocido. Probar cada plaza individualmente y después ambas juntas.

Prioridad realista: primero cables/conectores seguros, luego segundo divisor y segundo HC-SR04, después software de doble plaza y, por último, visualización y conectividad.

9. Encapsulado y despliegue propuesto

El Avance #2 solicita un modelo 3D conceptual, no necesariamente impreso. Se propone una carcasa que proteja ESP32, conexiones y alimentación, con aperturas para las dos bocas del HC-SR04, ventana visible para el indicador, paso del cable USB y acceso para mantenimiento. En una implementación real habría que fijar el sensor con orientación estable, protegerlo de impactos/humedad y evitar tirones del cableado.

No se han aportado medidas de carcasa ni un archivo Fusion 360. Antes de modelar deben medirse el ESP32, HC-SR04, display, conectores, protoboards o placa futura y espacio interno. Para transportar el Alpha actual se recomendó una caja de zapatos o cartón rígido acolchado, con USB desconectado; eso es protección temporal, no el encapsulado 3D exigido.

10. Conectividad y preparación de demostración

La guía exige transmisión de datos demostrable mediante Wi-Fi/Bluetooth u otra conectividad. Hasta ahora solo está documentado el Monitor Serie USB. Para cubrir este criterio se puede implementar una página web local alojada en el ESP32 que envíe “OCUPADO/LIBRE” y muestre las distancias, y verificar la recepción desde un teléfono u ordenador. No debe afirmarse que esto ya funciona.

Guion sugerido (5–7 min): problema y alcance (45 s); hardware y seguridad del divisor (60 s); prueba en vivo objeto cerca → rayita encendida / lejos → apagada (90 s); código y umbral (60 s); errores y mejoras, especialmente cables y segundo sensor (60 s); modelo de encapsulado y ampliación IoT (60–90 s). Todos los integrantes deben intervenir.

11. Checklist final para el equipo

Tarea	Estado documentado
Un sensor detecta ocupación y controla una rayita	Hecho y reportado funcionando.
Regla final: ocupado = encendido	Confirmada explícitamente por el usuario.
Segundo sensor funcional	Pendiente; recibido pero no cableado.
Modelo 3D conceptual con dimensiones y justificación	Pendiente.
Conectividad y transmisión de datos	Pendiente.
Funcionamiento continuo durante 5 minutos	Sin registro en el chat; realizar prueba.
Fotos, código comentado y diagrama actualizado	Código disponible aquí; anexos visuales y diagrama a terminar.
Demo ensayada 5–7 min / roles asignados	Pendiente de documentar.

12. Conclusión coherente para presentar

El equipo construyó un prototipo Alpha con ESP32 y un HC-SR04 que detecta un objeto cercano y enciende automáticamente una rayita LED para representar un estacionamiento ocupado; al alejar el objeto, la rayita se apaga. El proceso incluyó solución de problemas de programación, montaje y conexiones. La segunda unidad sensora está disponible como proyección inmediata, pero no se instaló por falta de cables macho-hembra y dificultades para acceder de forma segura a los pines del ESP32. El desarrollo demuestra una parte funcional de la solución de orientación de estacionamientos; faltan conectividad IoT, modelo 3D de encapsulado y pruebas prolongadas para satisfacer por completo el Avance #2.

Fuentes: guía “AVANCE #2 – Prototipo Alpha IoT Funcional” (TEI201, 10 páginas) subida por el usuario; conversaciones y fotografías del prototipo en este chat. Los pinouts y escenarios de expansión se presentan como propuestas que deben verificarse en el montaje real.